

Technical Specification Document

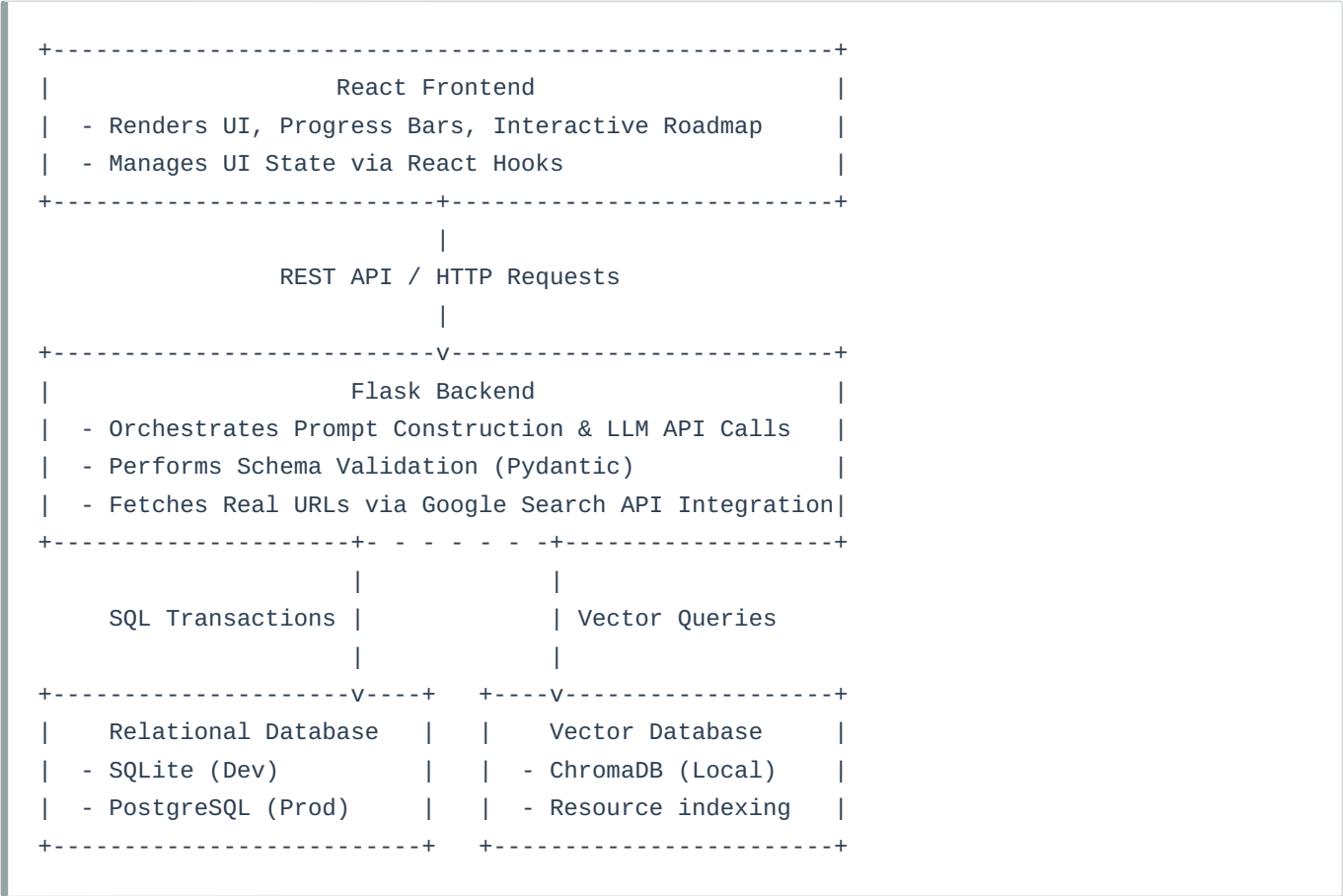
AI-Powered Personal Learning Path Generator

LEAD ARCHITECT: LAKSHAY

This comprehensive Technical Specification Document serves as the single source of truth for building the Personal Learning Path Generator. It outlines the architecture, data flows, exact requirements, and strategic mitigations for the challenges anticipated during development.

1. System Architecture & Tech Stack

The application uses a decoupled, client-server architecture designed for rapid development, clear data separation, and seamless AI integration.



Frontend

- **Framework:** React.js (Vite configuration) - Facilitates component-driven development for interactive roadmaps.
- **State Management:** React Context API or native useState hooks.
- **Styling:** Tailwind CSS.

Backend

- **Framework:** Flask (Python 3.10+) - Lightweight with native AI SDK compatibility.
- **Validation Layer:** Pydantic - Guarantees structural integrity of JSON from the LLM.

Database Layer

- **Relational DB:** SQLite (Dev) / PostgreSQL (Prod) - Stores user accounts, current progress, and saved paths.
- **Vector DB (Optional):** ChromaDB or Pinecone - Stores pre-indexed, verified educational resources to map to generated modules via semantic similarity search.

2. Technical & Functional Requirements

Functional Requirements (FR)

- **Curriculum Generation:** Users must be able to input any topic and receive a structured, chronological curriculum broken down by difficulty levels.
- **Resource Enrichment:** Every milestone must contain valid, external learning links (e.g., documentation, articles, videos).
- **Interactive Milestones:** Users must be able to check off completed modules, dynamically updating a visual progress bar.
- **Project-Based Learning:** Every generated path must include at least one practical project assignment matching the user's skill tier.

Non-Functional Requirements (NFR)

- **Type Safety & Validation:** The API contract between the frontend and backend must be strictly typed via JSON schemas.
- **Robust Error Handling:** The UI must display an elegant error state if the LLM generation times out or returns malformed text.
- **API Security:** AI API keys must remain strictly server-side; they must never be exposed to the client application.

3. Data Schema & Contracts

Target Curriculum Schema (Pydantic / Frontend Expectation):

```
{
  "topic": "String",
  "estimated_time": "String",
  "difficulty": "String",
  "curriculum": [
    {
      "phase": "Integer",
      "phase_title": "String",
      "milestones": [
        {
          "milestone_id": "String",
          "title": "String",
          "objectives": ["String"],
          "suggested_queries": ["String"],
          "resources": [
            {
              "resource_title": "String",
              "url": "String"
            }
          ],
          "milestone_project": {
            "project_title": "String",
            "description": "String"
          }
        }
      ]
    }
  ]
}
```

4. Anticipated Challenges & Strategic Mitigations

Risk / Challenge	Technical Impact	Engineering Mitigation
LLM Hallucinations	The model outputs broken, non-existent, or dead URLs for resources.	Decoupled Search: Instruct the LLM to output search queries instead of raw URLs. Intercept these queries in the Flask backend to call an external search API.
Connection Timeouts	Generating a complete roadmap takes >15 seconds, causing HTTP timeout errors.	Chunking or Streaming: Break down generation tasks. Generate the structure first, then lazily generate module details asynchronously.
Malformed JSON	The model returns plain text or broken JSON strings, crashing the UI parser.	Structured Output Modes: Use native SDK parameters like <code>response_mime_type: "application/json"</code> alongside Pydantic models.